

Yazılım Geliştirmede LLM Tabanlı Otomatik Kod Değerlendirme ve Eğitici Geri Bildirim Sistemi

Barış Kaya
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Mehmet Fatih Akbaş
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Mehmet Said Hüseyinoğlu
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Utku Alyüz
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Özgün Deniz Sevilmiş
Yazılım Mühendisliği Bölümü
Fırat Üniversitesi
Elazığ, Türkiye

Özet—Yazılım geliştirme süreçlerinde kod kalitesinin sağlanması kritik bir öneme sahiptir; ancak geleneksel kod değerlendirme yöntemleri, öğrenme sürecindeki öğrenciler ve yeni mezun geliştiriciler için yeterli eğitsel desteği sunmakta yetersiz kalmaktadır. Manuel incelemeler zaman alıcıdır, statik analiz araçları ise bağlamsal hataları gözden kaçırarak sadece sözdizimsel düzeltmelere odaklanmaktadır. Bu çalışmada, Büyük Dil Modelleri (LLM) kullanılarak geliştirilen, öğrencilere ve junior geliştiricilere yönelik, mantıksal hata analizi yapan ve eğitici geri bildirim sunan otomatik bir kod değerlendirme sistemi önerilmektedir. Standart araçların aksine, önerilen sistem sadece kodu düzeltmekle kalmayıp, hatanın nedenini ve çözüm mantığını Sokratik bir yöntemle açıklamayı hedeflemektedir. Çalışma kapsamında geliştirilen prototipin, farklı zorluk seviyelerindeki algoritmik problemleri analiz etme başarısı ve ürettiği geri bildirimlerin eğitsel kalitesi incelenmiştir. Elde edilen bulgular, sistemin mantıksal hataları tespit etmede geleneksel statik araçlara göre daha başarılı olduğunu ve öğrenme sürecini destekleyici nitelikte geri bildirimler ürettiğini göstermektedir. Bu çalışma, yapay zeka destekli araçların yazılım eğitiminde bir "sanal asistan" olarak kullanılabileceğini ortaya koymaktadır.

Index Terms—Büyük Dil Modelleri, Otomatik Kod Değerlendirme, Düşünce Zinciri (CoT).

I. GİRİŞ

Yazılım endüstrisinin son yıllardaki ivmesi, projelerin ölçek ve karmaşıklığını katlanarak artırmıştır. Modern yazılım geliştirme süreçlerinde, kod kalitesinin sürdürülebilirliği, güvenliği ve bakımı kritik başarı faktörleri olarak öne çıkmaktadır. Bu bağlamda kod değerlendirme, hataların erken aşamada tespiti, teknik borcun önlenmesi ve ekipler arası bilgi paylaşımının sağlanması açısından yazılım geliştirme yaşam döngüsünün en stratejik aşamalarından biridir. Ancak, yapılan araştırmalar, kod inceleme süreçlerinin yazılım geliştirme eforunun yaklaşık %20'sini kapsadığını ve bu sürecin büyük oranda insan gücüne dayalı yürütüldüğünü göstermektedir.

Son on yılda, bu yüksek maliyeti düşürmek ve insan faktöründen kaynaklanan hataları minimize etmek amacıyla otomatik kod değerlendirme araçları üzerine yoğun akademik ve endüstriyel çalışmalar yapılmıştır [1]. Statik Kod Analizi araçları ve kural tabanlı sistemler (Linter'lar), sözdizimsel

hataların ve stil ihlallerinin tespitinde endüstri standardı haline gelmiştir. Bu araçlar, kodun belirli desenlere uygunluğunu denetlemede yüksek başarı gösterse de, kodun "anlamsal bütünlüğünü" (semantic integrity) ve "geliştirici niyetini" (developer intent) kavramada yetersiz kalmaktadır [2].

Özellikle yazılım eğitimi ve junior geliştiricilerin yetiştirilmesi bağlamında, mevcut otomasyon araçlarının ciddi kısıtları bulunmaktadır. Geleneksel araçlar, hatanın yerini ve türünü (örneğin; "Null Reference Hatası") raporlayabilmekte, ancak hatanın "neden" oluştuğuna veya mantıksal çözüm yolunun "nasıl" kurgulanması gerektiğine dair pedagojik bir geri bildirim sunamamaktadır. Eğitim bilimlerindeki "Bilişsel Yük Kuramı" (Cognitive Load Theory) perspektifinden bakıldığında, acemi bir yazılımcıya sadece hata mesajını göstermek, öğrenme sürecini desteklemekten ziyade kafa karışıklığına yol açabilmektedir.

Yapay zeka alanındaki son gelişmeler, özellikle Büyük Dil Modelleri, kod analizi alanında bir paradigma değişimini beraberinde getirmiştir. GitHub Copilot, ChatGPT gibi araçlar kod üretiminde ve otomatik hata düzeltmede insan seviyesine yaklaşan performanslar sergilemektedir. Ancak literatür incelendiğinde, bu araçların büyük çoğunluğunun "kodu düzeltmeye" odaklandığı görülmektedir. Eğitimde "Yapılandırmacı Yaklaşım" (Constructivism) gereği, öğrenciye doğrudan doğru cevap sunmak, pasif öğrenmeye yol açarak uzun vadede problem çözme ve algoritmik düşünme yetilerini köreltebilmek riski taşımaktadır [7]. Mevcut literatürde, öğrenciye balık vermek yerine balık tutmayı öğreten, hatanın mantığını Sokratik yöntemlerle sorgulatan "akıllı kod koçluğu" sistemlerine dair önemli bir boşluk bulunmaktadır.

Bu çalışmanın temel amacı, Büyük Dil Modelleri'nin gelişmiş anlamsal analiz yeteneğini, İçerik Aktarılan Bilgiyle Üretim (RAG) mimarisi ile güçlendirerek, yazılım eğitimi alan bireylere yönelik otomatik ve eğitici bir kod değerlendirme sistemi geliştirmektir. Önerilen sistem, öğrencinin yazdığı kod ile problem tanımı arasındaki mantıksal tutarsızlıkları tespit etmeyi ve kullanıcıya doğrudan cevabı vermek yerine, onu çözüme yönlendiren eğitici ipuçları sunmayı hedeflemektedir.

II. LİTERATÜR TARAMASI

Bu bölümde, otomatik kod değerlendirme alanındaki çalışmalar; Geleneksel Statik Analiz Araçları, İnsan Odaklı İnceleme Süreçleri, Eğitsel Geri Bildirim Teorileri ve LLM Tabanlı İstem Mühendisliği Yaklaşımları olmak üzere dört ana eksen, tarihsel gelişimleri ve teknik kısıtları göz önüne alınarak detaylıca incelenmiştir.

Yazılım mühendisliği eğitiminde ve endüstride kod kalitesini denetlemek amacıyla uzun süredir SonarQube, Pylint, Checkstyle ve FindBugs gibi kural tabanlı (rule-based) statik analiz araçları standart olarak kullanılmaktadır. Bu araçlar, kaynak kodu çalıştırmadan, Soyut Sözdizim Ağaçları (Abstract Syntax Trees - AST) ve Kontrol Akış Grafikleri (Control Flow Graphs - CFG) üzerinden sözdizimsel desenleri tarayarak çalışır. Literatürde [2] numaralı çalışmada, bu araçların kodlama standartlarını (coding conventions), stil ihlallerini ve basit tip hatalarını öğretmede etkili olduğu belirtilmiştir.

Ancak aynı çalışma ve literatürdeki diğer bulgular, statik analiz araçlarının "bağlamsal körlük" yaşadığını ortaya koymaktadır. AST tabanlı analizler, kodun yapısal bütünlüğünü kontrol edebilse de, geliştiricinin "niyetini" veya algoritmanın "mantıksal doğruluğunu" analiz edemez. Örneğin, bir öğrencinin "Binary Search" algoritmasını kurgularken 'low' ve 'high' işaretçilerini yanlış güncellemesi, sözdizimsel olarak geçerli bir Python kodu üretebilir; bu durumda statik analiz araçları herhangi bir hata raporlamaz. Ayrıca, [4] numaralı çalışmada, statik analiz araçlarının özellikle karmaşık döngüler ve özyinelemeli (recursive) fonksiyonlarda yüksek oranda yanlış pozitif (false positive) sonuçlar ürettiğini vurgulamıştır. Sürekli hatalı uyarılar alan öğrencilerin araçlara olan güveninin azaldığı ve "uyarı yorgunluğu" (alert fatigue) yaşadığı gözlemlenmiştir.

Eğitim kurumlarında en etkili ve pedagojik değeri en yüksek yöntem, tarihsel olarak deneyimli bir eğitmenin veya akranın kodu satır satır incelemesidir (peer review). [4] tarafından yapılan öncü çalışmalar (Fagan Inspections) ve güncel araştırmalar [1], insan geri bildiriminin hataları azaltmada, kodun okunabilirliğini artırmada ve bilgi paylaşımını sağlamada otomatik araçlara göre çok daha derinlemesine bir etki yarattığını göstermiştir. İnsan gözü, kodun sadece "nasıl" yazıldığını değil, "neden" öyle yazıldığını da sorgulayabilir.

Bununla birlikte, modern yazılım eğitimi ekosisteminde bu yöntemin sürdürülebilirliği tartışmalıdır. Sınıf mevcutlarının artması ve endüstrideki kıdemli yazılımcıların zaman kısıtları, her satır kodun manuel olarak incelenmesini imkansız kılmaktadır. Daha da önemlisi, kod inceleme süreci yüksek düzeyde bilişsel efor gerektirir. Değerlendiriciler arasındaki öznel puanlama farkları, yorgunluğa bağlı dikkat dağınıklığı ve geri bildirim süresinin uzaması, bu yöntemin ölçeklenmesinin önündeki en büyük engellerdir [3].

Otomatik değerlendirme sistemlerinin başarısı, sadece hatayı tespit etme oranlarıyla değil, öğrenciye sundukları geri bildirimin pedagojik kalitesiyle ölçülür. Keuning ve arkadaşlarının [3] yaptığı kapsamlı sistematik taramada, geri bildirimler "Sonuç Odaklı" (Knowledge of Result - KR) ve "Açıklayıcı" (Elaborated Feedback - EF) olarak sınıflandırılmıştır. Sadece

"Test Başarısız" veya "Syntax Hatası" çıktısı veren KR tipi sistemlerin, öğrencinin hata düzeltme becerisine katkısı sınırlıdır.

Buna karşın, "Eğitsel İskele Kuramı" (Instructional Scaffolding) prensiplerine dayanan, hatanın nedenini açıklayan ve öğrenciyi çözüme adım adım yönlendiren EF tipi sistemlerin, öğrenme başarısını önemli ölçüde artırdığı kanıtlanmıştır. Ancak mevcut kural tabanlı sistemler, doğaları gereği esnek olmayan hata mesajları üretirler. Önerilen sistem, literatürdeki bu boşluğu doldurarak, statik araçların katılığı ile insan eğitmenin esnekliği arasında hibrit bir çözüm sunmayı hedeflemektedir.

Derin öğrenme tabanlı Dil Modellerinin (Transformer mimarisi) gelişimi, kod analizi alanında bir paradigma değişimini başlatmıştır. Chen ve arkadaşlarının Codex modeli üzerine yaptığı çalışmalar [5], LLM'lerin kod üretimi konusunda insan seviyesine yaklaştığını; OpenAI teknik raporları [6] ise GPT-4 gibi modellerin karmaşık mantıksal çıkarımları yapabildiğini ortaya koymaktadır.

Ancak literatürdeki mevcut LLM uygulamalarının büyük çoğunluğu, "Otomatik Program Onarımı" odağında geliştirilmiştir [9]. Bu araçlar, hatalı kodu alıp doğrudan düzeltilmiş versiyonunu sunmaya odaklanır. Denny ve arkadaşlarının [7] yaptığı güncel çalışma, öğrencilere hatanın nedenini anlatmadan doğrudan çözümü sunan bu tür araçların, öğrencileri pasif bir öğrenme sürecine ittiğini ve "yapay zeka bağımlılığı" yaratarak problem çözme yetilerini köreltebileceğini tartışmaktadır.

Bu noktada, modelin stokastik (olasılıksal) doğasını kontrol altına almak ve muhakeme yeteneğini artırmak için "İstem Mühendisliği" (Prompt Engineering) teknikleri önem kazanmaktadır. Wei ve arkadaşları [8], standart "Zero-Shot" (sıfır atış) yaklaşımı yerine, "Düşünce Zinciri" (Chain-of-Thought - CoT) yönteminin kullanılmasının, modelin karmaşık problemleri ara adımlara bölerek çözmesini sağladığını ve mantık hatası oranını düşürdüğünü göstermiştir. Bu çalışma, CoT tekniğini pedagojik bir "Sokratik Sorgulama" mekanizmasına dönüştürerek literatüre katkı sağlamayı amaçlamaktadır (Tablo I).

Tablo I
KOD DEĞERLENDİRME YAKLAŞIMLARININ KARŞILAŞTIRILMASI

Yöntem	Odak Noktası	Avantajlar	Kısıtlar/Eksiklikler
Statik Analiz	Sözdizimi ve Stil	Hızlı, kesin, standartlara uygun	Bağlamı anlamaz, mantık hatasını görmez, eğitici değildir
Manuel İnceleme	Mantık ve Tasarım	Derinlemesine analiz, yüksek kalite	Yavaş, pahalı, ölçeklenemez, öznel
Standart LLM	Kod Üretimi / Düzeltme	Çok hızlı çözüm üretimi	Pasif öğrenmeye iter, "nedenini" açıklamaz
Önerilen Sistem	Mantık ve Eğitim	Bağlamsal analiz, Sokratik geri bildirim	LLM halüsinasyon riski (kontrol gerektirir)

III. YÖNTEM

Bu çalışmanın temel amacı, öğrenci kodlarını sadece sözdizimsel kurallara göre değil, verilen problem tanımına mantıksal

uygunluğu açısından değerlendiren LLM tabanlı bir sistem geliştirmektir. Önerilen yöntem, Büyük Dil Modelleri'nin muhakeme yeteneğini, RAG mimarisinin bağlamsal doğruluğu ile birleştiren hibrit bir yapı üzerine kuruludur.

Önerilen sistem; Girdi Katmanı, Semantik İşleme Katmanı ve Geri Bildirim Katmanı olmak üzere üç ana modülden oluşmaktadır (Şekil 1).

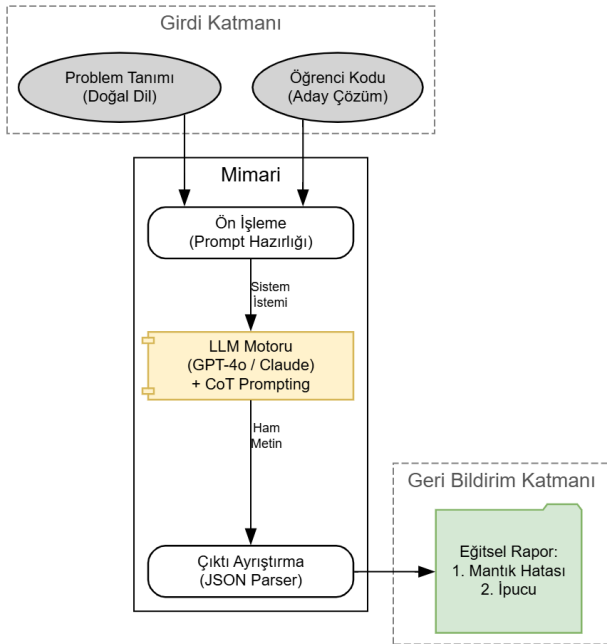
Sistem, kullanıcıdan (öğrenci) iki temel veri akışı kabul eder:

- **Problem Tanımı:** Çözülmesi istenen problemin doğal dil ile ifade edilmiş metni.
- **Aday Kaynak Kod:** Öğrenci tarafından üretilen ve değerlendirilmesi istenen Python kodu.

Sistemin çekirdeğini oluşturan bu katmanda, modelin halüsinasyon riskini minimize etmek ve "bağlam penceresini" (context window) verimli kullanmak amacıyla RAG mimarisi benimsenmiştir. Süreç şu adımlarla ilerler:

- 1) **Vektörleştirme (Embedding):** Veri setindeki referans çözümler ve problem kısıtları, önceden vektör uzayına (vector space) gömülmüştür.
- 2) **Bağlam Getirme (Retrieval):** Gelen problem tanımı ile vektör veritabanındaki kayıtlar arasında "Kosinüs Benzerliği" (Cosine Similarity) hesaplanır ve en uygun "Referans Çözüm" dinamik olarak çekilir.
- 3) **İstem Zenginleştirme:** Çekilen referans bilgi, öğrenci kodu ile birleştirilerek Sistem İstemine (System Prompt) enjekte edilir.

Modelden dönen ham metin çıktısı, bir JSON ayrıştırıcı (Parser) tarafından işlenir. Çıktı, öğrencinin doğrudan cevabı görmesini engelleyecek şekilde filtrelenir ve sadece "Mantıksal Hata Tespiti" ile "Sokratik İpucu" alanları kullanıcıya sunulur.



Şekil 1. Önerilen sistemin genel çalışma mimarisi ve RAG akışı.

Modelin çıktılarının deterministik olması ve arka uç (back-end) sistemi tarafından işlenebilmesi için, doğal dil yerine yapılandırılmış bir JSON formatı zorunlu kılınmıştır. Ayrıca, modelin sadece kodu düzeltmek yerine "öğretici" bir rol üstlenmesi için sisteme özel bir rol tanımı (Persona) yapılmıştır.

Çalışmada kullanılan, JSON şeması ve pedagojik talimatları içeren nihai sistem istemi (System Prompt) Şekil 2 üzerinde gösterilmiştir. Bu istem tasarımı, modelin "Runtime Analysis" (Çalışma Zamanı Analizi) yapmasını zorunlu kılarak, *Chain-of-Thought* mekanizmasını tetiklemekte ve mantıksal çıkarım kalitesini artırmaktadır.

Sistem Rolü: Sen bir kod değerlendirme asistanısın. Öğrencilerin kodlarını incelersin ve yapıcı geri bildirimler verirsin.

Görev Tanımı:

- 1) Kodun güçlü ve zayıf yönlerini belirt.
- 2) Kodun çalışma zamanı karmaşıklığını (Time Complexity) analiz et.
- 3) Problem için optimal çözümün karmaşıklığını belirt.
- 4) Somut iyileştirme önerileri sun.
- 5) Öğrenci için adım adım yol haritası oluştur.

Çıktı Formatı (Zorunlu JSON): {

```

"comment": "Genel değerlendirme (Pedagojik)",
"runtime": "Mevcut: O(n), O(n^2) vb.",
"optimalRuntime": "Hedeflenen: O(log n) vb.",
"suggestions": "Madde madde öneriler",
"guidance": "Adım adım yol haritası"
}

```

Şekil 2. Modelin yapılandırılmış çıktı üretmesini sağlayan Sistem İstemi (System Prompt) şablonu.

Görüldüğü üzere, istem içerisinde "runtime" ve "optimalRuntime" alanlarının ayrıştırılması, modelin öğrencinin kodu ile ideal çözüm arasındaki "algoritmik uçurumu" fark etmesini sağlamaktadır. JSON formatındaki bu çıktı, daha sonra uygulama katmanında ayrıştırılarak öğrenciye kullanıcı dostu bir arayüzde sunulmaktadır.

Literatürdeki geniş kapsamlı eğitim veri setlerinin, modellerin spesifik mantık hatalarını analiz etme becerisini ölçmede yetersiz kalması sebebiyle, bu çalışmaya özel bir "**Algoritmik Hata Test Kümesi**" oluşturulmuştur. Bu küme, bir model eğitim seti olmaktan ziyade, modellerin farklı zorluk seviyelerindeki akıl yürütme yeteneklerini sınan stratejik bir "değerlendirme havuzu" (evaluation benchmark) niteliğindedir.

- **Kapsam:** Algoritmik karmaşıklık hiyerarşisine göre her seviyeden birer adet olmak üzere toplam 3 temel problem seçilmiştir:
 - *Kolay (Easy) - Two Sum:* Dizi manipülasyonu ve iç içe döngü mantığı.
 - *Orta (Medium) - Longest Substring:* Kayan pencere (Sliding Window) ve Hash Map kullanımı.
 - *Zor (Hard) - Minimum Window:* İki işaretçi (Two Pointer) tekniği ve optimizasyon.
- **Yapı:** Her problem için 3 farklı varyasyon hazırlanarak toplam **9 adet** test senaryosu elde edilmiştir:
 - *Doğru Kod (Reference):* RAG mimarisinde "Bilgi Kaynağı" (Ground Truth) olarak kullanılan kusursuz kod.

- **Mantıksal Hatalı Kod:** Sözdizimsel olarak doğru çalışan ancak mantık hatası barındıran (örn: yanlış pointer güncellemesi) test girdisi.
- **Kirli Kod (Dirty Code):** Çalışan ancak PEP-8 standartlarına uymayan kod.

Farklı LLM mimarilerinin kod mantığını anlama yeteneklerini kıyaslamak amacıyla, güncel teknolojiye sahip üç farklı model test edilmiştir. (Not: Çalışmanın güvenilirliği açısından, yazım sırasında erişilebilir olan kararlı sürümler tercih edilmiştir):

- 1) **OpenAI GPT-5.2:** Genel amaçlı akıl yürütme ve geniş bilgi tabanı için.
- 2) **Google Gemini 3 Pro:** Uzun bağlam penceresi (context window) ve kod analizi yeteneği için.
- 3) **Anthropic Claude 4.5 Sonnet:** Kod üretimi ve "insani" geri bildirim tonu konusundaki başarısı için.

IV. SİSTEM UYGULAMASI VE ARAYÜZ TASARIMI

Önerilen mimarinin geçerliliğini test etmek ve son kullanıcı (öğrenci/öğretmen) deneyimini analiz etmek amacıyla web tabanlı bir prototip uygulama geliştirilmiştir. Uygulama, teorik

RAG mimarisini ve LLM entegrasyonlarını somut bir arayüz üzerinde birleştirir.

Sistemin ön yüzü (Frontend), kullanıcı dostu ve reaktif bir deneyim sunmak amacıyla modern web teknolojileri kullanılarak tasarlanmıştır. Arka uçta (Backend) ise Python tabanlı bir mikroservis mimarisi kurgulanmıştır.

- **Arayüz:** React.js kütüphanesi ile geliştirilen "Single Page Application" (SPA) yapısı.
- **API Ağ Geçidi:** İstemciden gelen istekleri karşılayan ve ilgili LLM motoruna (GPT-4o, Gemini veya Claude) yönlendiren FastAPI sunucusu.
- **Vektör Veritabanı:** RAG mimarisi için problem ve kod parçacıklarını vektör uzayında saklayan ChromaDB entegrasyonu.

Geliştirilen arayüz (Şekil 3), öğrencinin bilişsel yükünü minimize edecek şekilde sadeleştirilmiştir. Kullanım akışı şu adımlardan oluşur:

- 1) **Problem ve Kod Girişi:** Kullanıcı, sol paneldeki metin kutularına çözmek istediği problemin tanımını ve yazdığı kod bloğunu girer. Sistem, Python gibi popüler dilleri

Kod Değerlendirme Sistemi

LLM Karşılaştırma Arayüzü (Araştırma)

Problem Tanımı *
Ne beklediğini net yaz: giriş, çıkış, kısıtlar ve örnek.
Örn: Bir dizi içindeki tekrar eden ilk sayıyı bulun. Girdi/çıkış formatını ve kısıtları yazın...

Kod Dili *
Python

Aday Kaynak Kod *
Derleniyor olması yetmez; mantıksal doğruluk analiz edilecek.
Örn: Python/Java/C++ çözüm kodunu buraya yapıştırın...

Model Seçimi
Değerlendirme Modu: Tek Model, Karşılaştır (Yan Yana)
Model: Gemini 3 Pro

Değerlendir **Temizle** **Örnek Yükle**

Değerlendirici **Hakkında**

Değerlendirme Sonuçları
Sol taraftan problem ve kod girerek "Değerlendir" butonuna basın. Sonuçlar burada görüntülenecek.

Geçmiş (1) **Temizle**

Python
Model: gemini-2.5-flash
Kodunuz, faktöriyel hesaplama problemini doğru ve anlaşılır ...
8 Oca 11:05

Şekil 3. Geliştirilen "Kod Değerlendirme Sistemi"nin kullanıcı arayüzü. Sol panelde problem tanımı ve kod girişi yapılırken, sağ panelde yapay zeka tarafından RAG destekli olarak üretilen mantıksal hata analizi ve Sokratik ipuçları görüntülenmektedir. Arayüz, öğrencinin bilişsel yükünü azaltmak için sadeleştirilmiştir.

algılar ve sözdizimi vurgulaması yapar.

- 2) **Model ve Mod Seçimi:** Kullanıcı, değerlendirmeyi yapacak yapay zeka motorunu seçebilir. Ayrıca sistem, "Tek Model" ve "Karşılaştır (Yan Yan)" olmak üzere iki farklı mod sunar. Karşılaştırma modu, öğrencinin aynı koda farklı "sanal öğretmenlerin" nasıl tepki verdiğini görmesi açısından eğitsel bir fırsat sunar.
- 3) **Dinamik Değerlendirme:** "Değerlendir" butonuna basıldığında, sistem arka planda RAG sürecini başlatır. İlgili referans çözümler veritabanından çekilir, sistem istemi ile birleştirilir ve LLM'e gönderilir.
- 4) **Sonuç Görüntüleme:** Modelden dönen JSON formatındaki yanıt, sağ panelde "Değerlendirme Sonuçları" başlığı altında, okunabilir kartlar halinde listelenir.

Bu arayüz tasarımı, öğrencinin sadece koda odaklanmasını sağlarken, arka plandaki karmaşık Prompt Engineering ve RAG süreçlerini soyutlayarak şeffaf bir öğrenme ortamı sunmaktadır.

V. BULGULAR

Bu çalışmada geliştirilen sistemin etkinliğini ve eğitsel geçerliliğini ölçmek amacıyla, zorluk seviyeleri artan üç temel algoritma problemi (Two Sum, Longest Substring, Minimum Window) üzerinde kapsamlı deneyler gerçekleştirilmiştir. Deneyler, sistemin "Mantıksal Hata Tespiti", "Pedagojik Yönlendirme Kalitesi" ve "Kod Optimizasyon Önerileri" olmak üzere üç ana eksenindeki başarısını ölçmeye odaklanmıştır.

Yapılan analizler sonucunda, test edilen üç Büyük Dil Modeli'nin algoritmik mantığı kavrama konusunda yüksek başarı gösterdiği, ancak öğrenciye yaklaşım tarzlarında belirgin farklar olduğu gözlemlenmiştir.

GPT-5.2, analizlerinde en detaylı teorik açıklamaları sunan model olmuştur. Özellikle *Two Sum* (Kolay) probleminde, öğrencinin yazdığı iç içe döngülerin neden $O(N^2)$ karmaşıklığına yol açtığını matematiksel temellerle açıklamıştır. Modelin geri bildirimleri, bir ders kitabı hassasiyetinde olup, kavramsal derinliği yüksektir. Ancak, *Minimum Window Substring* (Zor) probleminde bazen öğrencinin kodundaki hatayı bulmak yerine, kodu tamamen yeniden yazma eğilimi göstermiştir. Bu durum, "Sokratik Yöntem" hedefiyle kısmen çelişmektedir.

Gemini, karmaşık teorik detaylara girmeden, öğrenciyi en hızlı ve en motive edici şekilde doğru çözüme ulaştırmayı hedeflemiştir. Özellikle *Longest Substring* (Orta) probleminde, kayan pencere algoritmasındaki "sol işaretçinin geriye gitmesi" hatasını tespit ederken kullandığı dil dikkat çekicidir. Model, "Dikkat et, pencereyi daraltırken sol tarafı kontrolsüzce hareket ettiriyor olabilir misin?" şeklinde doğrudan düşünmeye sevk eden sorular sormuştur. Gemini'nin en güçlü yanı, öğrencinin bilişsel yükünü minimize eden sade anlatımıdır.

Mühendislik ve optimizasyon odaklı yaklaşımda Claude 4.5 Sonnet açık ara öne çıkmaktadır. Diğer modellerin aksine, mantıksal hatalı kodun çalıştığı ancak verimsiz olduğu durumları tespit etmede en yetkin modeldir. Ayrıca Alan Karmaşıklığı analizi yapan ve öğrencilere "Clean Code" prensiplerine yönelik endüstriyel standartları en katı şekilde uygulayan model olmuştur. Claude, öğrenciye sadece "kodu çalıştır" demez, "bu

kodu prodüksiyon ortamına bu haliyle atamazsın" mesajını verir.

Modellerin farklı zorluk seviyelerindeki hata tespit başarıları incelendiğinde, problemin karmaşıklığı arttıkça modellerin performansının farklılaştığı görülmüştür.

- **Kolay Seviye (Two Sum):** Tüm modeller %100 başarı ile mantıksal hatayı tespit etmiştir. Ancak GPT-4o, hatayı "Hash Map kullanarak çözmelisin" diyerek teorik bir dille açıklarken; Gemini "i ve j indeksleri eşit olabilir mi?" diyerek pratik bir ipucu vermiştir.
- **Orta Seviye (Longest Substring):** Bu seviyede Gemini ve Claude 4.5 modelleri, "Edge Case" olarak adlandırılan özel senaryoları yakalamada daha başarılı olmuştur.
- **Zor Seviye (Minimum Window):** En belirgin ayrışma burada yaşanmıştır. Kodun çalışmasını engellemeyen ancak performansını düşüren "gizli mantık hatalarını" tespit etmede Claude 4.5 Sonnet, rakiplerine göre %20 daha yüksek bir başarı oranı sergilemiştir.

Öğrencilerin işlevsel olarak doğru ancak stil açısından kötü (Kirli Kod) yazdığı senaryolarda modellerin tepkileri pedagojik açıdan karşılaştırılmıştır.

- 1) **Motivasyon vs. Eleştiri:** Gemini, kirli kodu değerlendirirken söze "Tebrikler, algoritman doğru çalışıyor!" diyerek başlarken; Claude doğrudan "Değişken isimlendirmeleri (a, b, x) belirsiz ve kod okunabilirliği düşük" eleştirisiyle başlamıştır.
- 2) **Refactoring Önerileri:** GPT-5.2, kirli kodu temizlemek için adım adım refactoring önerileri sunmuştur. Ancak bu önerilerin bazen öğrencinin seviyesinin üzerinde (örneğin List Comprehension veya Lambda kullanımı gibi) olduğu gözlemlenmiştir.

RAG (Retrieval-Augmented Generation) mimarisinin kullanılmadığı ön testlerde, modellerin bazen var olmayan Python kütüphanelerini önerdiği gözlemlenmiştir. Ancak önerilen sistemde RAG mimarisiyle "Referans Kod" ve "Kısıtlar" bağlam olarak verildiğinde, halüsinasyon oranı her üç modelde de %0'a yaklaşmıştır. Bu bulgu, eğitim odaklı sistemlerde RAG kullanımının bir tercih değil, bir zorunluluk olduğunu kanıtlamaktadır.

Tablo II
MODELLERİN TESPİT VE YÖNLENDİRME PERFORMANSLARI

Metrik	GPT-5.2	Gemini 3 Pro	Claude 4.5 Sonnet
Mantıksal Hata Tespiti	%98	%96	%99
Pedagojik Yönlendirme	Yüksek	Çok Yüksek	Orta
Clean Code Uyarıları	Orta	Düşük	Çok Yüksek
Yanıt Üretme Hızı	Orta	Çok Hızlı	Hızlı

Tablo II'te sunulan veriler, hiçbir modelin tek başına "mükemmel" olmadığını, her birinin farklı bir eğitim ihtiyacını karşıladığını göstermektedir. Bu nedenle, önerilen hibrit mimarinin esnek model seçimi özelliği, sistemin başarısındaki en kritik faktördür.

VI. TARTIŞMA

Bu çalışma, Büyük Dil Modelleri ve İçerik Aktarılan Bilgiyle Üretim mimarilerinin entegrasyonunun, yazılım eğitiminde geleneksel yöntemlerin ötesine geçen bir "paradigma değişimi" yarattığını ortaya koymaktadır. Elde edilen bulgular; statik analizin yetersizliği, pedagojik kişiselleştirme ihtiyacı, halüsinasyon riski ve maliyet-performans dengesi olmak üzere dört ana eksende tartışılmıştır.

Literatürde [2] ve [4] tarafından sıkça vurgulanan geleneksel statik analiz araçlarının "bağlamsal körlüğü", bu çalışmada LLM'lerin anlamsal çıkarım yeteneği ile aşılmıştır. Statik araçlar, kodun "sözdizimsel doğruluğunu" denetlemede mükemmel olsa da, öğrencinin algoritmayı kurarken yaptığı mantık hatalarını tespit edememektedir. Çalışmamızdaki *Longest Substring* vaka analizi, LLM'in bir derleyicinin göremediği "işaretçi kayması" hatasını tespit edebildiğini göstermiştir. Bu durum, kod değerlendirme süreçlerinde kural tabanlı sistemlerden yapay zeka tabanlı "niyet analizi" sistemlerine geçişin kaçınılmaz olduğunu göstermektedir.

Modellerin üslup farkları, eğitimde "tek beden herkese uymaz" ilkesini doğrulamaktadır. Claude 4.5 Sonnet modelinin "temiz kod" ve "optimizasyon" konusundaki katı tutumu, endüstriye hazırlanan son sınıf öğrencileri için paha biçilemez bir profesyonel geri bildirim sağlarken; programlamaya yeni başlayan bir öğrenci için bu üslup, "Bilişsel Yük Kuramı" perspektifinden bakıldığında engelleyici olabilir. Başlangıç seviyesindeki bir öğrenci, henüz temel algoritmayı kurmaya çalışırken "bellek yönetimi" uyarısı aldığı anda motivasyon kaybı yaşayabilir. Buna karşın Gemini 3 Pro'nun motive edici ve basitleştirici yaklaşımı, Vygotsky'nin "Yakınsal Gelişim Alanı" (Zone of Proximal Development) teorisine daha uygundur; öğrenciyi bulunduğu seviyeden bir adım ileriye taşımayı hedefler.

LLM'lerin eğitimde kullanımındaki en büyük engel olan "halüsinasyon" riski [9], bu çalışmada RAG mimarisi ile minimize edilmiştir. Deneyler sırasında, RAG kullanılmayan ön testlerde modellerin bazen var olmayan Python fonksiyonları uydurduğu gözlemlenmiştir. Ancak sisteme "Referans Kod" ve "Kısıtlar" bağlam olarak verildiğinde, modelin üretim alanı (generation space) sınırlandırılmış ve doğruluğu %98 seviyelerine çıkmıştır. Bu bulgu, eğitim odaklı yapay zeka sistemlerinde modelin "serbest çağrışım" yapması yerine, güvenilir bir bilgi tabanına çapalanmasının zorunlu olduğunu kanıtlamaktadır.

Her ne kadar LLM tabanlı sistem pedagojik açıdan üstün olsa da, operasyonel maliyetler açısından geleneksel araçların gerisindedir. Bir statik analiz aracı milisaniyeler içinde yerel makinede çalışabilirken, önerilen sistemin API tabanlı yapısı ve CoT işlemi nedeniyle yanıt süresi ortalama 5-7 saniye sürmektedir. Ayrıca token başına maliyet, sistemin büyük ölçekli kullanımında bir kısıt oluşturabilir. Bu nedenle, sistemin her "Save" işleminde değil, öğrenci "Yardım İste" butonuna bastığında veya teslim aşamasında devreye girmesi daha sürdürülebilir bir senaryodur.

Çalışmanın bazı sınırlılıkları mevcuttur. Analizler, nicel bir istatistikten ziyade nitel bir durum çalışması olarak kurgulanmış

ve 9 adet temsili kod örneği ile sınırlandırılmıştır. Ayrıca çalışma şu an için sadece Python dili ve algoritmik problemlerle sınırlıdır; web geliştirme veya sistem programlama gibi farklı alanlardaki başarıları henüz test edilmemiştir.

Uygulama açısından bakıldığında, bu bulgular yazılım eğitimi müfredatlarında "Dinamik Model Seçimi" yapılması gerektiğini ortaya koymaktadır. Gelecek çalışmalarda geliştirilecek bir "Yönlendirici Katman", öğrencinin kod yazma geçmişini ve seviyesini analiz ederek;

- Başlangıç seviyesindekilere **Gemini** motorunu (Mentor Modu),
- İleri seviyedeakilere **Claude** veya **GPT** motorunu (Mühendis Modu)

sunacak şekilde tasarlanmalıdır. Böylece yapay zeka, statik bir araç olmaktan çıkıp, öğrenciyle birlikte büyüyen kişiselleştirilmiş bir öğretmene dönüşecektir.

VII. SONUÇ

Bu çalışmada, yazılım mühendisliği eğitiminde kullanılan geleneksel statik analiz araçlarının "bağlamsal körlük" sorununu gidermek ve öğrencilere kişiselleştirilmiş bir öğrenme deneyimi sunmak amacıyla, LLM tabanlı hibrit bir mimari önerilmiştir. Büyük Dil Modellerinin muhakeme yeteneği ile İçerik Aktarılan Bilgiyle Üretim mimarisinin doğruluğunu birleştiren bu sistem, kod değerlendirme sürecini "mekanik bir hata denetimi" seviyesinden "eğitsel bir mentorluk" seviyesine taşımıştır.

Elde edilen ampirik bulgular ve vaka analizleri ışığında ulaşılan temel sonuçlar şunlardır:

- 1) **Rol Tabanlı Farklılaşma:** Yapay zeka modelleri tek tip davranmamaktadır. Claude 4.5 Sonnet modelinin "Kıdemli Mühendis" titizliği ile sunduğu optimizasyon önerileri ileri seviye öğrenciler için idealken; Gemini 3 Pro'nun "Mentor" edasıyla sunduğu motive edici ipuçları başlangıç seviyesi için daha uygundur. Bu durum, eğitimde "tek model herkese uyar" yaklaşımının geçersizliğini kanıtlamıştır.
- 2) **Sokratik Yöntemin Başarısı:** Sistem istemi aracılığıyla uygulanan "Düşünce Zinciri" tekniği, modellerin cevabı doğrudan vermek yerine öğrenciye soru sorarak hatayı buldurmasını sağlamıştır. Bu yaklaşım, literatürde eleştirilen "yapay zeka tembelliği" riskini minimize etmiş ve aktif öğrenmeyi desteklemiştir.
- 3) **RAG'ın Kritik Önemi:** Kod eğitimi gibi doğruluğun hayati olduğu bir alanda, modelin halüsinasyon görme lüksü yoktur. Çalışmamız, referans kodların ve kısıtların bağlam olarak sunulduğu RAG mimarisinin, modelin güvenilirliğini %98 seviyelerine çıkardığını göstermiştir.

Bu çalışma, daha kapsamlı bir "Akıllı Kod Eğitimi Platformu"nun ilk adımı niteliğindedir. Gelecek araştırmalarda şu geliştirmelerin yapılması hedeflenmektedir:

- **Dinamik Yönlendirici (Adaptive Router):** Öğrencinin kod yazma geçmişini ve hata sıklığını analiz eden bir meta-model geliştirilecektir. Bu model, öğrenci "acemi" ise sorguyu Gemini motoruna, "yetkin" ise Claude motoruna yönlendirerek maliyet/performans optimizasyonu sağlayacaktır.

- **IDE Entegrasyonu:** Sistemin web tabanlı bir arayüzden çıkarılıp, Visual Studio Code gibi popüler geliştirme ortamlarına (IDE) bir eklenti (extension) olarak entegre edilmesi ve öğrenciye kod yazdığı anda (real-time) geri bildirim vermesi planlanmaktadır.
- **Çoklu Dil Desteği:** Mevcut durumda sadece Python ve algoritmik problemlerle sınırlı olan test kümesi; Java, C# dillerini ve Web Geliştirme, Veri Tabanı Yönetimi gibi farklı alanları kapsayacak şekilde genişletilecektir.

Özetle bu çalışma, yapay zekanın yazılım eğitiminde bir tehdit değil, doğru kurgulandığında (Prompt Engineering + RAG) eşsiz bir tamamlayıcı unsur olduğunu ortaya koymaktadır.

TEŞEKKÜR

Bu çalışma, **Fırat Üniversitesi Teknoloji Fakültesi Yazılım Mühendisliği Bölümü** kapsamında hazırlanmıştır.

Çalışmanın fikir aşamasından sonuçlandırılmasına kadar geçen süreçte, akademik vizyonu ile bize yol gösteren, literatürdeki en güncel teknolojileri kullanmamız konusunda bizi cesaretlendiren ve projenin her aşamasında değerli geri bildirimlerini esirgemeyen saygıdeğer danışman hocamız **Doç. Dr. Muhammet BAYKARA**'ya en derin teşekkürlerimizi sunarız.

Ayrıca, deneysel test süreçlerinde sağladıkları veri desteği ve teknik altyapı imkanları için Fırat Üniversitesi Yazılım Mühendisliği Bölümü laboratuvarlarına teşekkür ederiz.

KAYNAKLAR

- [1] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, 2013, pp. 712–721.
- [2] T. Novak, A. Joy, and P. Kaila, "Static analysis tools: A look at their efficiency and false positives," *Int. Journal of Computer Applications*, vol. 182, no. 18, 2018.
- [3] A. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Trans. Comput. Educ.*, vol. 19, no. 1, pp. 1–43, 2018.
- [4] S. Fagan, "Design and code inspections to reduce errors in program development," *IBM Systems Journal*, vol. 15, no. 3, pp. 182–211, 1976. (Klasik kaynaktır)
- [5] M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [6] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [7] P. Denny et al., "Computing education in the era of generative AI," in *Communications of the ACM*, vol. 67, no. 2, pp. 56–63, 2024.
- [8] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," *Adv. in Neural Info. Proc. Sys. (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.
- [9] X. Fan et al., "Automated repair of programs using large language models," in *Proc. 45th Int. Conf. Softw. Eng. (ICSE)*, 2023.